# ITA06 – MACHINE LEARNING

# ASSIGNMENT 1

## DESIGN, ANALYSE AND VALIDATE AN INTELLIGENT EARLY-WARNING DECISION-SUPPORT SYSTEM FOR ACADEMIC RISK IDENTIFICATION

*Using Concept Learning and Decision Tree Learning*

Student Name: A. Lakshmi Devi

Register Number: 192525176

Programme / Branch: AI & ML (Artificial Intelligence & Machine Learning)

Course Code: ITA0611

Course Name: Machine Learning

Assignment: 1

Total Marks: 50

Duration: 1 Week

## CONTENTS

1. Scenario and dataset summary
2. Task 1 – ML problem formulation
3. Task 2 – Learning paradigm analysis
4. Task 3 – Concept learning
5. Task 4 – Candidate Elimination
6. Task 5 – Version space and inductive bias
7. Task 6 – ID3 decision tree
8. Task 7 – Heuristic search
9. Task 8 – Overfitting and underfitting
10. Task 9 – Python implementation and tests
11. Task 10 – Experimental analysis and recommendation
12. Task 11 – Day-4 design challenge
13. Conclusion
14. Appendix – Reproducible Python implementation

## SCENARIO AND DATASET SUMMARY

The university wants to identify students who may fail by using information collected after Week 6. The output is AtRisk = Yes or No. A reliable warning allows the academic office to arrange counselling, remedial classes and mentoring. The model must be interpretable so that advisors can explain each warning.

| Dataset | Records | Attributes | Role |
|---|---|---|---|
| A | 5 | Attendance, Assignments, StudyHours, PriorCGPA, LMS, Mentoring | Concept learning / Candidate Elimination |
| B | 14 | Attendance, Assignments, LMS, PriorCGPA | ID3 decision-tree training |
| T | 6 | Same four attributes as B | Held-out test set |
| C | At least 500 required | Discretised or synthetic | Scalability and noise experiments |

Important data limitation: Dataset C is specified by the assignment but no actual public file or synthetic records were supplied. The small datasets A, B and T are fully analysed below. Dataset C experiments are specified with a reproducible generator; performance values for C must be generated by running the supplied code and must not be presented as measured results before execution.

## TASK 1 – IDENTIFY AND FORMULATE THE ML PROBLEM (3 MARKS)

### 1.1 T–P–E formulation

Task (T): Given a student's categorical information after Week 6, predict whether the student will be AtRisk at the end of the semester.

Experience (E): The labelled historical records in Dataset A and Dataset B, where the academic council has assigned AtRisk = Yes or No.

Performance (P): At-risk recall is the primary measure because missing an at-risk student is costly. Also report precision, F1-score, confusion matrix, overall accuracy and false-negative count.

### 1.2 Attributes and relevance

| Attribute | Type / values | Role and relevance | Potential concern |
|---|---|---|---|
| Attendance | Categorical: Low, Medium, High | Low attendance can indicate missed teaching and assessments. | May reflect illness, disability, transport or approved absence. |
| Assignments | Categorical: Irregular, Regular | Late/missing work may indicate disengagement or difficulty. | Submission pattern may depend on course design. |
| StudyHours | Categorical: Low, High | Study time may reflect preparation and engagement. | Self-reported and potentially inaccurate. |
| PriorCGPA | Categorical: Weak, Strong | Prior performance is a useful academic baseline. | May encode historical disadvantage; not a complete measure of ability. |
| LMS | Categorical: Passive, Active | Shows online learning engagement. | Low activity may occur when students use offline materials. |
| Mentoring | Categorical: No, Yes | Can indicate support participation. | Selection effect: students already at risk may be more likely to receive mentoring. |

### 1.3 Target concept and instance space

Let x = (a, s, h, g, l, m) be an instance. The target concept is c(x) = 1 when the academic council labels the student AtRisk = Yes; otherwise c(x) = 0.

Dataset A cardinalities are $3 \times 2 \times 2 \times 2 \times 2 \times 2 = 96$ possible instances. Five examples cover $5/96 \times 100 = 5.2083\%$ of the space.

Dataset B cardinalities are $3 \times 2 \times 2 \times 2 = 24$ possible instances. Fourteen examples cover $14/24 \times 100 = 58.3333\%$ of the space.

The task is supervised binary classification because the historical examples contain input attributes and council-provided Yes/No labels.

### 1.4 ML issues and mitigations

| Issue | Connection to this application | Mitigation |
|---|---|---|
| Small sample | Only 5 examples in A and 14 in B. | Collect more semesters; use cross-validation and report uncertainty. |
| Label noise | Different advisors may disagree about risk. | Define a common rubric; audit disagreements; use adjudicated labels. |
| Sampling bias | Historical students may not represent new cohorts or online learners. | Check subgroup and cohort coverage; monitor drift. |
| Missing / inaccurate inputs | StudyHours is self-reported; LMS activity may be incomplete. | Validate fields, record missingness and avoid treating missing as low. |
| Fairness | Attendance and CGPA may proxy for circumstances outside student control. | Conduct subgroup error analysis and human review. |
| Privacy | Academic performance is sensitive student data. | Role-based access, minimisation, encryption and retention limits. |
| Generalisation | Sparse training coverage leaves unseen combinations. | Use held-out cohorts and uncertainty / manual review. |

A false negative is costlier than a false positive: a missed at-risk student receives no timely support. Therefore emphasise recall for AtRisk = Yes, while monitoring precision so that excessive false alarms do not overload advisors.

## TASK 2 – ANALYSE THE LEARNING PARADIGM (3 MARKS)

| Criterion | Supervised | Unsupervised | Reinforcement |
|---|---|---|---|
| Labels | Available: AtRisk Yes/No. | Not required; labels ignored during grouping. | Reward feedback, not ordinary class labels. |
| Output here | Risk class and probability / score. | Behavioural clusters or profiles. | Policy for selecting interventions. |
| Feedback | Compare predictions with council labels. | Cluster quality and advisor interpretation. | Reward after actions and outcomes. |
| Data needed | Labelled student records. | Student attributes, labelled or unlabelled. | Repeated state-action-outcome sequences. |
| Evaluation | Recall, precision, F1, calibration and confusion matrix. | Cluster stability, usefulness and separation. | Long-term student outcomes and intervention cost. |
| Interpretability | Decision-tree rules can be shown. | Cluster descriptions need interpretation. | Policy can be harder to explain. |
| Main risk | Noisy labels and overfitting. | Clusters do not directly mean AtRisk. | Unsafe exploration and confounding. |

Concrete unsupervised use: cluster students into profiles such as low attendance/low LMS activity, high attendance/irregular assignments, and consistently engaged students. These profiles can help advisors plan group support. Clustering cannot directly provide a validated AtRisk = Yes/No decision because cluster IDs are not the council's target labels.

A possible reinforcement-learning follow-up: state = student risk profile and past interventions; action = schedule mentoring, remedial class or counselling; reward = improved course completion and attendance minus intervention cost. RL is unsuitable for the initial classification because there is no sequential action-reward dataset and experimenting with support decisions could harm students.

Select supervised learning for the core system. Strongest objection: the labelled sample is small and noisy. Rebuttal: use a transparent baseline, expand the audited dataset, validate on later cohorts, calibrate thresholds and keep advisors in the decision loop.

Hybrid proposal: supervised decision tree for risk prediction plus unsupervised clustering to identify common student-support profiles. The tree makes the individual risk decision; clusters help tailor interventions.

## TASK 3 – FORMULATE THE CONCEPT LEARNING TASK (3 MARKS)

Target concept c: X → {0,1}, where c(x)=1 means AtRisk=Yes. Positive examples are E1, E2 and E4; negative examples are E3 and E5.

| Example | Hypothesis tuple (Attendance, Assignments, StudyHours, PriorCGPA, LMS, Mentoring) | Class |
|---|---|---|
| E1 | ⟨Low, Irregular, Low, Weak, Passive, No⟩ | Positive |
| E2 | ⟨Low, Irregular, Low, Strong, Passive, No⟩ | Positive |
| E3 | ⟨High, Regular, High, Strong, Active, Yes⟩ | Negative |
| E4 | ⟨Low, Irregular, High, Weak, Passive, Yes⟩ | Positive |
| E5 | ⟨Medium, Irregular, High, Strong, Active, No⟩ | Negative |

Hypothesis symbols: a specific value restricts an attribute; ? accepts any value; Ø denotes no value / the most specific initial constraint. Most general hypothesis is ⟨?,?,?,?,?,?⟩. Most specific is ⟨Ø,Ø,Ø,Ø,Ø,Ø⟩. E1 and E3 can be written as their exact six-value tuples above.

For each attribute with d values, there are d + 1 meaningful choices (each value or ?). Therefore the number of semantically distinct conjunction hypotheses is (3+1)(2+1)^5 = 4 × 243 = 972. Including the always-false empty hypothesis gives 973 semantic hypotheses. The instance space has 96 points. Under a syntax that permits Ø independently in each position, there are (3+2)(2+2)^5 = 5 × 1024 = 5120 syntactic tuples; many of these denote the same empty set, so semantic count is much smaller.

Example ordering: h1=⟨Low,?, ?, ?, ?, ?⟩ is more general than h2=⟨Low,Irregular,?, ?, ?, ?⟩. h3=⟨?, ?,Low,?, ?, ?⟩ is incomparable with h1 and h2 because it constrains a different attribute and neither covers the other's full set.

| Hypothesis | Result on Dataset A | Reason |
|---|---|---|
| ha = ⟨Low,?,?,?,?,Passive,?⟩ | Inconsistent | It covers E1, E2 and E4 (positive) but also no negative examples; actually consistent with the supplied data. |
| hb = ⟨?,Irregular,?,?,?,?,?⟩ | Inconsistent | It covers E1, E2, E4 (positive) and E5 (negative), so E5 contradicts it. |
| hc = ⟨Low,Irregular,Low,?,?,?⟩ | Consistent | It covers E1 and E2 only; it covers neither E3, E4 nor E5. |

A hypothesis satisfies an instance if the instance matches all its specified attribute constraints. It is consistent with the training data only if it classifies every positive example positive and every negative example negative. For example, ha satisfies E1; consistency is checked across all five examples.

## TASK 4 – APPLY CANDIDATE ELIMINATION (6 MARKS)

Initial boundaries: S0 = {⟨Ø,Ø,Ø,Ø,Ø,Ø⟩}; G0 = {⟨?,?,?,?,?,?⟩}. Positive examples minimally generalise S; negative examples minimally specialise members of G that cover them.

| Step | Example | Label | Action / resulting S | Resulting G |
|---|---|---|---|---|
| 0 | — | — | S0 = {⟨Ø,Ø,Ø,Ø,Ø,Ø⟩} | G0 = {⟨?,?,?,?,?,?⟩} |
| 1 | E1 | Yes | S1={⟨Low,Irregular,Low,Weak,Passive,No⟩} | G1={⟨?,?,?,?,?,?⟩} |
| 2 | E2 | Yes | Generalise PriorCGPA: S2={⟨Low,Irregular,Low,?,Passive,No⟩} | G2={⟨?,?,?,?,?,?⟩} |
| 3 | E3 | No | S3 unchanged | G3={⟨?,?,?,?,?,No⟩, ⟨Low,?,?,?,?,?⟩, ⟨?,Irregular,?,?,?,?⟩, |

| | | | | |
|---|---|---|---|---|
| | | | | ⟨?,?,Low,?,?,?⟩, ⟨????,Passive,?⟩} |
| 4 | E4 | Yes | Generalise StudyHours and Mentoring: S4={⟨Low,Irregular,?, ?,Passive,?⟩} | Remove G members that fail to cover E4; G4={⟨Low,?,?,?,?,?⟩, ⟨?,Irregular,?,?,?,?⟩, ⟨?,?,?,?,Passive,?⟩} |
| 5 | E5 | No | S5 unchanged | G5={⟨Low,?,?,?,?,?⟩, ⟨?,?,?,?,Passive,?⟩} |

Explanation of E3: the universal G hypothesis covers this negative instance, so minimally specialise one attribute at a time to a value different from E3 while remaining at least as general as S2. The useful boundary specialisations are Low attendance, Irregular assignments, Low study hours, Passive LMS and Mentoring=No. The PriorCGPA specialisation is not retained because S2 already has ? in that position and the generated boundary must remain at least as general as S.

E4 is positive, so S2 is minimally generalised: StudyHours Low becomes ?, and Mentoring No becomes ?. The candidate G member ⟨Mentoring=No⟩ is removed because it does not cover E4. E5 is negative; G members that cover E5 are specialised. The Irregular-only member is removed because E5 has Irregular and no valid minimal specialisation can remain at least as general as S5. The final boundaries are S5={⟨Low,Irregular,?,?,Passive,?⟩} and G5={⟨Low,?,?,?,?,?⟩, ⟨?,?,?,?,Passive,?⟩}.

Note: the displayed tuple for the Passive-only G member is ⟨?,?,?,?,Passive,?⟩. Candidate Elimination is order-sensitive; the table follows the stated E1→E5 sequence and the standard minimal-specialisation rule.

## TASK 5 – VERSION SPACE AND INDUCTIVE BIAS (4 MARKS)

The final version space contains six hypotheses lying between S5 and G5:

- ⟨Low, Irregular, ?, ?, Passive, ?⟩
- ⟨Low, Irregular, ?, ?, ?, ?⟩
- ⟨Low, ?, ?, ?, Passive, ?⟩
- ⟨Low, ?, ?, ?, ?, ?⟩
- ⟨?, Irregular, ?, ?, Passive, ?⟩
- ⟨?, ?, ?, ?, Passive, ?⟩

Version-space size = 6. The hypothesis ⟨?,Irregular,?,?,?,?,?⟩ is not in the version space because it covers negative example E5 (Irregular assignments, AtRisk=No).

| Query | Votes Yes | Votes No | Decision |
|---|---|---|---|
| Q1 = ⟨Low,Regular,High,Weak,Passive,No⟩ | 2 | 4 | No by majority; disagreement exists. |
| Q2 = ⟨Low,Irregular,Low,Strong,Passive,Yes⟩ | 6 | 0 | Yes; unanimous. |

Q1 is ambiguous because hypotheses that constrain Attendance=Low or LMS=Passive vote Yes, while the other hypotheses do not cover it. Q2 is unanimously positive. The most informative next labelled query is one on which the version-space hypotheses disagree; Q1 is therefore the better query. More formally, choose an instance that divides the six hypotheses as evenly as possible.

| Term | Meaning in this assignment |
|---|---|
| Training data | The five labelled examples E1–E5. |
| Hypothesis space | All candidate conjunctions representable by the value/? format. |
| Version space | The subset of hypotheses consistent with every labelled example. |
| Inductive bias | The restriction to conjunctions of attribute-value constraints. |

Candidate Elimination assumes the true concept is representable in the chosen conjunctive hypothesis language and labels are consistent. Adding the new positive example ⟨High,Regular,High,Strong,Passive,No⟩ conflicts with the previous negative E3, which is identical in all attributes except Mentoring=Yes versus No? E3 is High, Regular, High, Strong, Active, Yes, so it is not identical; the new example is not a direct contradiction. It is not covered by S5 and requires generalisation, while the final G boundary may be reduced or version space altered. If the hypothesis space were the full power set of the 96-point instance space, the number of Boolean concepts would be 2^96, an enormous space. With no restriction on concept form, finite data cannot determine labels for unseen points; an inductive bias is needed to generalise.

## TASK 6 – CONSTRUCT THE DECISION TREE USING ID3 (6 MARKS)

Dataset B contains 14 records: 7 Yes and 7 No. $H(B) = -(7/14)\log_2(7/14) - (7/14)\log_2(7/14) = 1.0000$ bit.

| Root attribute | Partitions (Yes/No) | Weighted entropy | Information gain |
|---|---|---|---|
| Attendance | Low 4/1; Medium 2/3; High 1/3 | 0.8364 | 0.1636 |
| Assignments | Irregular 5/3; Regular 2/4 | 0.9389 | 0.0611 |
| LMS | Passive 5/2; Active 2/5 | 0.8631 | 0.1369 |
| PriorCGPA | Weak 6/1; Strong 1/6 | 0.5917 | 0.4083 |

Root = PriorCGPA because it has the highest information gain, 0.4083.

### 6.1 Recursive splits

For PriorCGPA=Strong (7 records: 1 Yes, 6 No), entropy=0.5917. Attendance gain=0.3060, Assignments gain=0.1281 and LMS gain=0.1981. Choose Attendance.

Strong & Attendance=Low contains two records (one Yes, one No). Assignments and LMS both give gain 1.0000. Tie-break alphabetically by attribute name, so choose Assignments. Irregular→Yes; Regular→No.

Strong & Attendance=Medium is pure No (3 records); Strong & Attendance=High is pure No (2 records).

For PriorCGPA=Weak (7 records: 6 Yes, 1 No), entropy=0.5917. Attendance gain=0.3060; Assignments and LMS each give 0.1981. Choose Attendance.

Weak & Attendance=Low is pure Yes (3); Weak & Attendance=Medium is pure Yes (2). Weak & Attendance=High contains one Yes and one No; Assignments and LMS each give 1.0000, so alphabetical tie-break selects Assignments. Irregular→Yes; Regular→No.

### 6.2 Final tree and rules

```
PriorCGPA?
├── Strong: Attendance?
│   ├── High   → No
│   ├── Medium → No
│   └── Low: Assignments?
│       ├── Irregular → Yes
│       └── Regular   → No
└── Weak: Attendance?
    ├── Low    → Yes
    ├── Medium → Yes
    └── High: Assignments?
        ├── Irregular → Yes
        └── Regular   → No
```

Rules: (1) If PriorCGPA=Strong and Attendance=High, predict No. (2) Strong and Medium→No. (3) Strong and Low and Assignments=Irregular→Yes. (4) Strong and Low and Regular→No. (5) Weak and Low or Medium→Yes. (6) Weak and High and Irregular→Yes. (7) Weak and High and Regular→No.

Stopping criteria: stop when a node is pure or no attributes remain. Ties are resolved alphabetically. Tree depth is 3 edges; it has 7 decision nodes including root/internal nodes? Counting root + two attendance nodes + two assignment nodes = 5 decision nodes, and 8 leaves, so 13 total nodes. Training accuracy = 14/14 = 100%.

## TASK 7 – HEURISTIC SEARCH IN DECISION TREE LEARNING (3 MARKS)

ID3 search formulation: initial state is a single root containing all 14 training examples; operator selects an unused attribute and splits the current node by its values; heuristic is information gain; termination occurs when a node is pure, attributes are exhausted or a stopping rule applies.

At the root, four attributes are evaluated. At the two PriorCGPA child nodes, three attributes each are evaluated. At the two mixed terminal candidate nodes, two attributes each are evaluated. Total gain computations = 4+3+3+2+2 = 14.

At the root, PriorCGPA IG=0.4083 and Attendance IG=0.1636. Split information for PriorCGPA is 1.0000 because it divides 7/7; gain ratio=0.4083. Attendance split information = $-(5/14 \log_2(5/14)+5/14 \log_2(5/14)+4/14 \log_2(4/14))$ ≈1.5774; gain ratio≈0.1037. Gain ratio penalises attributes that fragment data into many small groups by dividing information gain by split information.

A unique StudentID creates 14 singleton partitions, each pure, so weighted entropy=0 and information gain=1.0000. It is a misleading perfect training split: it memorises records and cannot generalise to a new student. ID3 therefore must exclude identifiers.

Alternative Attendance-rooted tree: Attendance root; within each value use the best remaining gain. This root has lower gain (0.1636) than PriorCGPA (0.4083), and subsequent splits are more complex. A greedy tree chooses the largest immediate gain and never backtracks to reconsider earlier choices.

Advantages: fast, interpretable local decisions; naturally handles categorical attributes. Limitations: greedy choice may miss a globally smaller tree; small data and noisy labels can create unstable splits.

## TASK 8 – OVERFITTING AND UNDERFITTING (3 MARKS)

M0 predicts the majority class. Dataset B is tied 7–7; deterministic tie rule predicts No. Training accuracy=7/14=50%. Test accuracy=3/6=50% (T1, T2 and T5 are No).

M1 is the depth-1 root-only split. With the root PriorCGPA, Strong has 1 Yes/6 No→No and Weak has 6 Yes/1 No→Yes. Training accuracy=12/14=85.71%. On T: T1 No correct, T2 Yes wrong, T3 Yes correct, T4 Yes correct, T5 No correct, T6 Yes correct: 5/6=83.33%.

M2 fully grown tree from Task 6 has training accuracy 14/14=100%. Its predictions on T using the learned rules are T1 Yes, T2 Yes, T3 No, T4 Yes, T5 No, T6 Yes; 3/6=50%.

| Model | Depth | Training accuracy | Test accuracy | Diagnosis |
|---|---|---|---|---|
| M0 majority | 0 | 50.00% | 50.00% | Underfits; no attribute split. |
| M1 max depth 1 | 1 | 85.71% | 83.33% | Best observed generalisation on this tiny test set. |
| M2 full tree | 3 | 100.00% | 50.00% | Overfits; perfect training fit but poor test result. |

Causes: Dataset B has only 14 examples over 24 possible combinations; some leaves contain one or two students; one unusual label can change a branch. A full tree memorises these small groups.

Controls: maximum depth=1 and minimum samples per split=4. A validation/pruning procedure should be used rather than choosing solely from six test cases. A simple depth-1 pruned tree is the M1 model above, with 85.71% training and 83.33% test accuracy. For tied leaves, predict the parent-node majority; if still tied, choose No as the conservative deterministic tie rule.

Select a shallow/pruned tree provisionally, but six test cases are too few for reliable claims. Improve evaluation using stratified 5-fold cross-validation on larger multi-cohort data, a later-semester holdout, confidence intervals and subgroup recall.

## TASK 9 – IMPLEMENT AND TEST THE ML TECHNIQUES (5 MARKS)

The following implementation uses custom hypothesis coverage, generality, Candidate Elimination, entropy, information gain and recursive ID3. It does not call a library Candidate Elimination or decision-tree learner. The test table gives expected checks; observed outputs should be confirmed by running the code on the submitted computer.

```
# Python 3.x – ITA06 Machine Learning Assignment 1
import math
from collections import Counter, defaultdict

DOMAINS_A = [
    ["Low", "Medium", "High"], ["Irregular", "Regular"],
    ["Low", "High"], ["Weak", "Strong"],
    ["Passive", "Active"], ["No", "Yes"]
]
A = [
 ("Low","Irregular","Low","Weak","Passive","No","Yes"),
 ("Low","Irregular","Low","Strong","Passive","No","Yes"),
 ("High","Regular","High","Strong","Active","Yes","No"),
 ("Low","Irregular","High","Weak","Passive","Yes","Yes"),
 ("Medium","Irregular","High","Strong","Active","No","No")
]
B = [
 ("Low","Irregular","Passive","Weak","Yes"),
 ("Low","Irregular","Passive","Strong","Yes"),
 ("Low","Regular","Active","Weak","Yes"),
 ("Low","Regular","Active","Strong","No"),
 ("Medium","Irregular","Passive","Weak","Yes"),
 ("Medium","Irregular","Active","Strong","No"),
 ("Medium","Regular","Passive","Weak","Yes"),
 ("Medium","Regular","Active","Strong","No"),
 ("High","Irregular","Passive","Weak","Yes"),
 ("High","Irregular","Active","Strong","No"),
 ("High","Regular","Passive","Strong","No"),
 ("High","Regular","Active","Weak","No"),
 ("Medium","Irregular","Passive","Strong","No"),
 ("Low","Irregular","Active","Weak","Yes")
]
T = [
 ("Low","Irregular","Active","Strong","No"),
 ("Medium","Regular","Active","Weak","No"),
 ("High","Regular","Passive","Weak","Yes"),
 ("Medium","Irregular","Active","Weak","Yes"),
 ("Low","Regular","Passive","Strong","No"),
 ("High","Irregular","Active","Weak","Yes")
]

def covers(h, x):
    return all(a == "?" or a == v for a, v in zip(h, x))

def more_general(h1, h2):
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def minimal_generalise(s, x):
    return tuple(v if a == "∅" else
                 (a if a == v or a == "?" else "?")
                 for a, v in zip(s, x))

def minimal_specialise(g, x, s, domains):
    out = set()
    for i, (a, value, domain) in enumerate(zip(g, x, domains)):
        if a == "?":
```

```
            for v in domain:
                if v != value:
                    h = list(g); h[i] = v; h = tuple(h)
                    if more_general(h, s):
                        out.add(h)
    return out

def candidate_elimination(data, domains):
    S = {tuple("∅" for _ in domains)}
    G = {tuple("?" for _ in domains)}
    for row in data:
        x, label = row[:-1], row[-1]
        if label == "Yes":
            G = {g for g in G if covers(g, x)}
            S = {minimal_generalise(s, x) if not covers(s, x) else s
                 for s in S}
            S = {s for s in S if any(more_general(g, s) for g in G)}
        else:
            S = {s for s in S if not covers(s, x)}
            new_g = set()
            for g in G:
                if covers(g, x):
                    for s in S:
                        new_g |= minimal_specialise(g, x, s, domains)
                else:
                    new_g.add(g)
            G = {g for g in new_g if not any(
                g != h and more_general(h, g) for h in new_g)}
        print("S =", sorted(S), "G =", sorted(G))
        if not S or not G:
            print("EMPTY VERSION SPACE")
            break
    return S, G

def entropy(labels):
    n = len(labels)
    if n == 0: return 0.0
    counts = Counter(labels)
    return -sum((v/n) * math.log2(v/n) for v in counts.values())

def information_gain(rows, j):
    parent = entropy([r[-1] for r in rows])
    groups = defaultdict(list)
    for r in rows: groups[r[j]].append(r[-1])
    weighted = sum(len(v)/len(rows) * entropy(v)
                   for v in groups.values())
    return parent - weighted

def id3(rows, attributes, names, max_depth=None, depth=0,
        min_samples_split=2):
    labels = [r[-1] for r in rows]
    counts = Counter(labels)
    majority = sorted(counts, key=lambda k: (-counts[k], k))[0]
    if len(counts) == 1 or not attributes:
        return ("leaf", majority)
    if max_depth is not None and depth >= max_depth:
        return ("leaf", majority)
    if len(rows) < min_samples_split:
        return ("leaf", majority)
    gains = [(information_gain(rows, j), j) for j in attributes]
    best_gain = max(g for g, j in gains)
    # deterministic alphabetical tie-break by attribute name
    j = min(j for g, j in gains if abs(g-best_gain) < 1e-12)
    branches = {}
    for value in sorted(set(r[j] for r in rows)):
        subset = [r for r in rows if r[j] == value]
        branches[value] = id3(subset, [a for a in attributes if a != j],
                              names, max_depth, depth+1, min_samples_split)
    return ("node", j, majority, branches)
```

```
def predict(tree, x):
    if tree[0] == "leaf": return tree[1]
    _, j, majority, branches = tree
    if x[j] not in branches: return majority
    return predict(branches[x[j]], x)

if __name__ == "__main__":
    print("CANDIDATE ELIMINATION")
    candidate_elimination(A, DOMAINS_A)
    names = ["Attendance", "Assignments", "LMS", "PriorCGPA"]
    print("ROOT INFORMATION GAINS")
    for j, name in enumerate(names):
        print(name, round(information_gain(B, j), 4))
    tree = id3(B, list(range(4)), names)
    print("TREE:", tree)
    print("TRAIN:", sum(predict(tree,r[:-1]) == r[-1] for r in B), "/", len(B))
    print("TEST:", [(predict(tree,r[:-1]), r[-1]) for r in T])
```

| Test | Input | Expected behaviour | Result to verify |
|---|---|---|---|
| TC1 | Dataset A E1–E5 | Print S and G after each row; final S/G as Task 4. | Compare console trace with hand trace. |
| TC2 | Entropy([]) / pure labels / 7 Yes + 7 No | 0.0000 / 0.0000 / 1.0000. | Run entropy unit tests. |
| TC3 | Dataset B, four root attributes | Attendance .1636; Assignments .0611; LMS .1369; PriorCGPA .4083. | Compare printed gains. |
| TC4 | Train on B | Classify all 14 rows; training accuracy 14/14 for fully grown tree. | Check rule output and training count. |
| TC5 | Predict T | Compare six predictions with Task 8. | Record exact matches and accuracy. |
| TC6 | Contradictory labels | Candidate Elimination reports empty version space rather than crashing. | Add opposite label for identical input. |
| TC7 | Attendance=Very Low | Unseen category uses parent majority or explicit manual review. | Test fallback policy. |
| TC8 | Equal information gains | Alphabetical attribute name resolves tie consistently. | Repeat run and compare tree. |

Reproducibility: use Python 3.x and the same A/B/T tuples above. Save the code as assignment1.py and run `python assignment1.py`. The deterministic tie-break removes run-to-run variation. For Dataset C generation, fix a random seed and document the rule and noise rate.

## TASK 10 – EXPERIMENTAL ANALYSIS, THEORY VS EXPERIMENT, RECOMMENDATION (6 MARKS)

### 10A. Dataset characteristics

| Dataset | N | Attributes | Class balance | Instance-space coverage |
|---|---|---|---|---|
| A | 5 | 6 | 3 Yes / 2 No | 5/96 = 5.2083% |
| B | 14 | 4 | 7 Yes / 7 No | 14/24 = 58.3333% |
| T | 6 | 4 | 3 Yes / 3 No | Held-out; not added to training coverage |
| C | ≥500 | 4 (aligned to B) | Generator-dependent | Calculate unique combinations / 24 |

Dataset C generator specification: sample each of the four categorical attributes independently using a fixed seed. Define the clean target rule before adding noise, for example AtRisk=Yes if (Attendance=Low and Assignments=Irregular) OR

(PriorCGPA=Weak and LMS=Passive); otherwise No. Then flip a configurable percentage of labels (0%, 5%, 10%). This is a synthetic benchmark, not real university evidence.

### 10B. Test metrics on T

Using the fully grown Task 6 tree and the stated deterministic tree, predictions are [Yes, Yes, No, Yes, No, Yes] for T1–T6. With AtRisk=Yes as positive: TP=2 (T4,T6), FN=1 (T3), FP=2 (T1,T2), TN=1 (T5). Accuracy=3/6=0.5000; precision=2/4=0.5000; recall=2/3=0.6667; F1=2×0.5×0.6667/(0.5+0.6667)=0.5714.

| | **Predicted Yes** | **Predicted No** |
|---|---|---|
| Actual Yes | TP=2 | FN=1 |
| Actual No | FP=2 | TN=1 |

The assignment asks for leave-one-out or 5-fold cross-validation on B+T (20 records). The supplied tiny dataset has repeated and contradictory patterns; execute the implementation with a fixed fold assignment and report the resulting metrics. Do not treat the six-row test score as a reliable population estimate.

### 10C. Learning curve, complexity and noise

Learning-curve experiment: for sizes 4, 6, 8, 10, 12 and 14, sample training subsets from B (and then larger sizes from C), train the tree and record training and validation accuracy. Repeat each size over several fixed-seed samples and plot mean accuracy. Complexity experiment: train with maximum depth 0, 1, 2, 3 and unrestricted; record training/validation accuracy, node count and leaf count.

Noise experiment: generate Dataset C with 0%, 5% and 10% label flips. For each noise level compare a fully grown tree with a controlled tree (maximum depth and minimum samples per split=4). Record accuracy and tree size. Expected theoretical behaviour is that the fully grown tree becomes more complex and sensitive to noisy labels; this is a hypothesis to test, not a measured result.

### 10D. Candidate Elimination vs ID3

| **Aspect** | **Candidate Elimination** | **Decision tree** |
|---|---|---|
| Representation | Conjunctions of attribute constraints. | Hierarchical attribute tests and class leaves. |
| Strength | Maintains all consistent hypotheses as S/G boundaries. | Can represent disjunctive rules and local interactions. |
| Noise | A single contradiction can collapse the version space. | Can tolerate some noise with stopping and pruning, though still affected. |
| Unseen values | No native category unless domain is extended. | Needs explicit default/unknown branch policy. |
| Interpretability | Boundary hypotheses are compact but can be multiple. | IF–THEN paths are easy to explain. |
| Data needs | Assumes noise-free consistent labels and a suitable hypothesis language. | Needs labelled examples; benefits from more data and pruning. |

The theory predicts Candidate Elimination is fragile under contradictory labels and restrictive conjunction bias. ID3 can model disjunctions but greedy information gain and small leaves can overfit. The small Dataset B experiment supports the overfitting concern: full tree training accuracy is 100% while test accuracy is 50%; depth-1 training/test accuracy is 85.71%/83.33%. This test sample is too small to establish a general performance ranking.

### 10E. Evidence-based recommendation

For the original small-data prototype, use a shallow, pruned and interpretable decision tree as an advisor-support tool, not an autonomous decision. It can express rules that Candidate Elimination's conjunction-only hypothesis language cannot represent. Candidate Elimination is useful pedagogically when labels are consistent and the target is expected to be a conjunction, but it is unsuitable for noisy real-world advisor labels.

The final selection must be revalidated on a larger, audited cohort. Optimise the decision threshold for at-risk recall, inspect precision and workload, provide reasons for every flag, and allow advisor override. The full tree is not selected because it overfits the 14-record training sample.

## TASK 11 – DAY-4 DESIGN CHALLENGE (8 MARKS)

### 11A. Impact analysis

| New condition | Original assumption affected | Area | Consequence |
|---|---|---|---|
| About 2,000 labelled records; weekly arrivals | Small fixed dataset | Data / algorithm | Can train and validate richer models; needs incremental data governance. |
| 5–10% inconsistent labels | Noise-free labels | Data / algorithm | Candidate Elimination may collapse; tree splits may fit errors. |
| Very Low attendance and unseen combinations | Fixed attribute values | Hypothesis space / algorithm | Unknown category handling and drift monitoring required. |
| At least 90% recall | Accuracy-only evaluation | Evaluation | Threshold and class weighting must prioritise recall. |
| Human-readable explanations required | Simple output assumed | Algorithm / deployment | Every warning needs a traceable reason and advisor review. |

N2 is a previously observed feature combination with a label that may contradict a learned rule; a noise-tolerant tree can absorb it as a minority exception or handle it through pruning/weights. A noise-free Candidate Elimination learner treats an inconsistent label as a hard contradiction and may eliminate all hypotheses. E6 is especially important because it has the same six attributes as E1 but the opposite label (E1=Yes, E6=No). No deterministic noise-free concept over these attributes can satisfy both; CE's version space must become empty.

### 11B. Re-evaluation

Candidate Elimination on E1–E5 followed by E6: E6 is identical to E1 in attributes but has the opposite class, so the version space collapses to empty. E7 and E8 should be processed cumulatively after E1–E5 as required; they expand or constrain the boundary only if the current version space is non-empty. With contradictory E6 included, a strict noise-free CE system cannot recover without removing or relabelling an example.

Decision tree update with N1–N3: N1 introduces Attendance=Very Low, an unseen category. A categorical tree cannot follow a learned branch; use a documented fallback such as the parent-node majority plus an uncertainty flag, or map the value to a clinically/academically justified ordered bin only after domain approval. N2 and N3 add further positive examples and can alter gains, depth and leaves. Rebuild the tree using the complete B+N dataset and report the actual resulting structure by running the code.

The old model cannot guarantee 90% recall merely from the small original dataset. It must be retrained and threshold-validated. Explanations can be generated from the decision path, e.g., "Flagged because PriorCGPA=Weak and Attendance=Low"; the wording must match the actual trained path.

### 11C. Adaptation / redesign

Decision: MODIFY the original design into a noise-aware, recall-constrained, explainable decision-support pipeline. Retain the interpretable tree and rule display; replace strict Candidate Elimination as the operational learner.

- Use a decision tree with minimum samples per leaf, pruning and a deterministic unseen-value fallback.
- Audit advisor labels; adjudicate disagreements and retain a confidence/consensus label where possible.
- Use class weights or a lower decision threshold to achieve at least 90% recall on validation data; expect precision to decrease as more students are flagged.
- Consider a hybrid of a pruned tree and calibrated ensemble only if its recall benefit is demonstrated and explanations remain available.

- Use an explicit OTHER/UNKNOWN category for new values and trigger manual review until enough examples accumulate.
- Generate human-readable explanations from the path and show the key input values, confidence/uncertainty and recommended support action.
- Monitor cohort drift, recall, precision, false negatives, subgroup performance, advisor overrides and label disagreements each term.

### 11D. Evidence-based defence

The concept-learning inductive bias is too restrictive when labels conflict and the true concept may be disjunctive. Candidate Elimination assumes a noise-free target and is directly contradicted by E1/E6. A decision tree supports disjunctions and local conditions, and controlled growth reduces overfitting. The 90% recall requirement changes the optimisation target: accuracy alone is insufficient. The final threshold must be selected on a representative validation cohort, and its precision/workload trade-off must be reported.

Remaining limitation: advisor labels may still be inconsistent and student behaviour may change across cohorts. Monitor monthly label disagreement and feature distributions; trigger review and retraining when recall falls below 90% or drift exceeds a pre-agreed threshold. Human review remains mandatory for high-impact decisions.

## CONCLUSION

The assignment shows how concept learning and decision-tree learning differ in representation, inductive bias and response to inconsistent labels. Candidate Elimination is useful for a small, noise-free conjunctive concept, but its version space collapses when E6 contradicts E1. ID3 selects PriorCGPA at the root with information gain 0.4083 and fits the training set, but the full tree generalises poorly on the six test records. A controlled tree is a better provisional design, and the Day-4 conditions justify modifying it into a noise-aware, recall-focused and explainable decision-support system. All deployment decisions must be validated on a larger, audited Dataset C or real university data.

## APPENDIX – DATASET C GENERATOR AND EXPERIMENT NOTES

```
# Reproducible synthetic Dataset C generator (500+ records)
import random, csv
random.seed(42)
rows = []
for i in range(600):
    attendance = random.choice(["Low", "Medium", "High"])
    assignments = random.choice(["Irregular", "Regular"])
    lms = random.choice(["Passive", "Active"])
    cgpa = random.choice(["Weak", "Strong"])
    # Documented clean target rule:
    risk = ((attendance == "Low" and assignments == "Irregular")
            or (cgpa == "Weak" and lms == "Passive"))
    rows.append([attendance, assignments, lms, cgpa,
                 "Yes" if risk else "No"])
# To create a noisy copy, flip labels at 5% or 10%:
def add_label_noise(data, rate, seed=42):
    rng = random.Random(seed)
    noisy = [row[:] for row in data]
    for row in noisy:
        if rng.random() < rate:
            row[-1] = "No" if row[-1] == "Yes" else "Yes"
    return noisy
with open("dataset_C_600.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["Attendance","Assignments","LMS","PriorCGPA","AtRisk"])
    w.writerows(rows)
```

Submission note: run the implementation and generator, paste the actual console outputs and graphs into the report, and clearly label synthetic experiments as synthetic. The uploaded brief (pages 1–14) supplies the scenario, datasets, task rubrics and changed Day-4 requirements used throughout this answer.